

93478 - Computer Vision Project

By: Toh Kok Soon



Content

- Paper Presentation - Masked Autoencoders Are Scalable Vision Learners
 - Introduction
 - Language vs Vision
 - Related Work
 - MAE Architecture
 - Result
 - Discussion
- Experiment - Masked Autoencoders vs Standard Vision Transformer for Basic Classification Problem
 - Introduction to Experiment
 - Scope of Experiment
 - Methodology
 - Data
 - Training Setup
 - Transformations
 - Results
 - Conclusion

Masked
Autoencoders are
Scalable Vision
Learners

Introduction

- Motivation:
 - Models capacity is higher now, easily overfit larger amount of data, therefore demanding even more data
 - Data are hard to come by especially labelled data
 - Addressed in NLP using self-supervised learning such as masked autoencoding in BERT

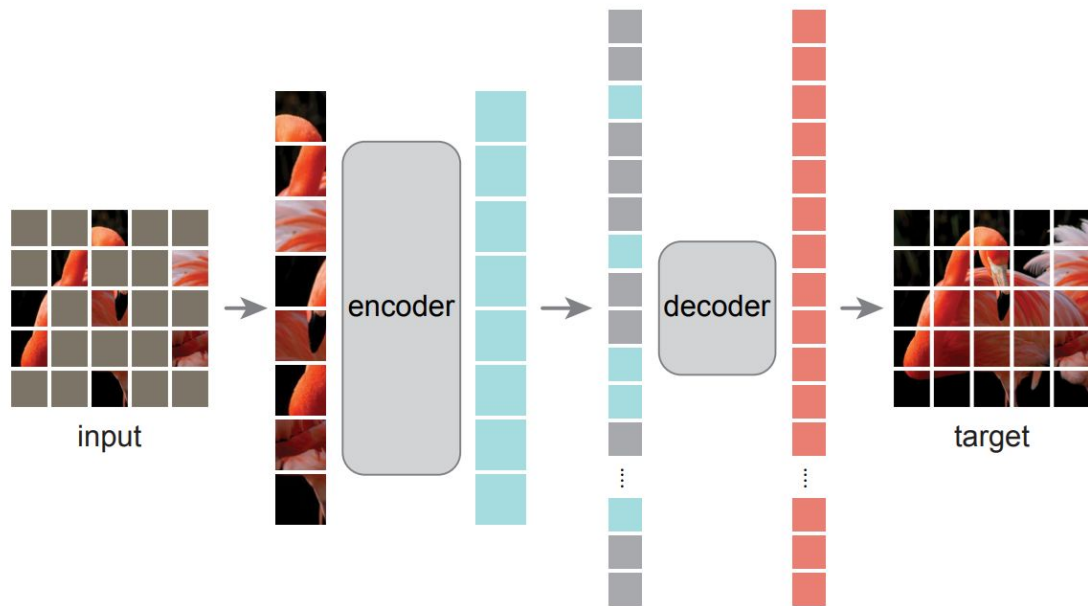
Differences in Language and Vision

- Architecture
 - Vision model are predominantly CNN up to only recently
 - Not straightforward to use mask tokens or positional embedding like in transformers
 - Vision transformers are starting to become more popular
- Information Density
 - Words are semantically dense, removing a few of them changes meaning drastically
 - Images are less so, they can be inferred from neighbouring regions
 - Paper address this by increasing the masking ratio
- Decoder
 - Decoder for words are much easier than decoder for images(i.e. predicting word vs reconstructing image patches)
 - Decoder design is much more important in images than in language

Related Work

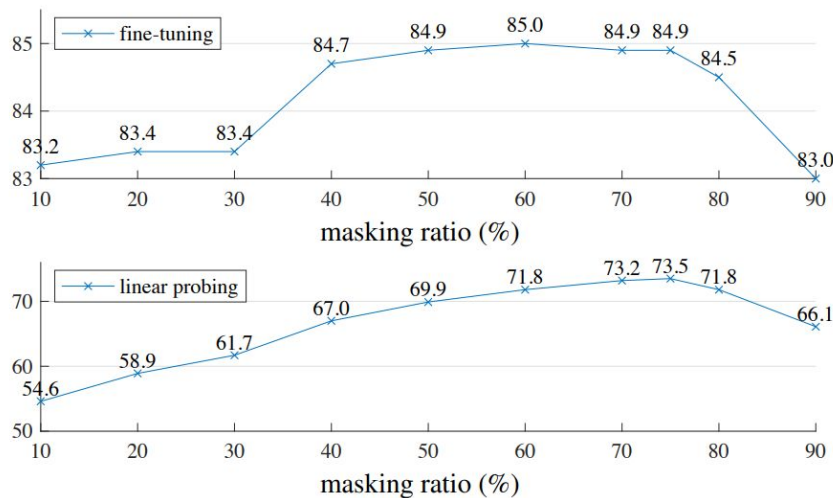
- Masked language modeling
 - Randomly hide tokens in texts
 - Model learn to predict the hidden text
- Autoencoding
 - Model compresses input into a representation, then reconstructs the original input.
 - Learns important features without needing labels.
- Masked image encoding
 - Randomly masks parts of an image (patches).
 - Model learns to predict/reconstruct the missing image regions.
 - Forces the model to understand visual patterns and object structure.
 - This is the key to masked autoencoder but prior work were not as successful
- Self-supervised learning
 - Model learns from data without human-provided labels.
 - Creates its own learning task from the input data

Masked Autoencoder Architecture



Architecture: Masking

- Sample a subset of non-overlapping patches and remove remaining ones
- High masking ratio up to a certain point improves the model performance in downstream tasks



Architecture: Encoder

- Encoder is a standard vision transformer
- Only takes in unmasked patches
 - No masked tokens
 - Reduce training time and memory
- Encoder purpose is to learn the representations that would be used for downstream tasks

Architecture: Decoder

- Takes in encoded visual patches as well as mask tokens
- Positional embedding is added to every token
- The decoder outputs a reconstructed image and a mean squared error(MSE) between the reconstructed image and the original image is used for computing the loss
 - Only masked patches are included in this computation
- The decoder is only used for pre-training and will be discarded for downstream tasks

Result

- Comparison of top-1 accuracy against ViT-Large
- 76.5 -> 82.5 with regularization
- MAE still performs better

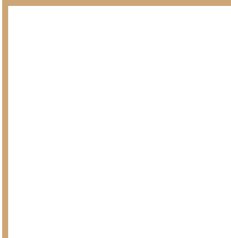
scratch, original [16]	scratch, our impl.	baseline MAE
76.5	82.5	84.9

Advantages of MAE Over Standard ViT

- Semi-supervised - Don't require labelled data until working with downstream tasks
- Scalable - faster training time and less memory consumption
- Improved Accuracy

My Interpretation of the Paper

- From the result, it can be theorized that Masked autoencoders act as a strong regularization technique for training Visual Transformer
 - Lead to better robustness
 - Lead to better generalization accuracy



Experiment

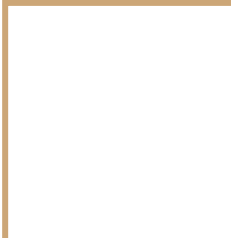


Introduction to Experiment

- Claim by the paper:
 - Less labelled data required for pre training
 - Better accuracy
- My own interpretation
 - Better robustness

Scope of Experiment

- Effects of MAE is mostly seen in pre-training
 - Unrealistic for us due to limited hardware and time
- Goal of Experiment:
 - To see if MAE ViT is more robust than regular supervised ViT
 - To see if MAE ViT is more accurate than regular supervise ViT

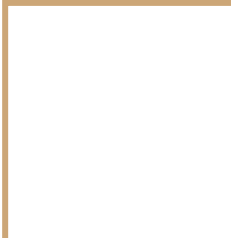


Methodology



Methodology (Experiment Setup)

- Fine-tuning a supervised ViT (baseline) and a MAE ViT (mae)
 - End to end fine tuning to simplify experiment
- Running evaluation on various corruption transformations:
 - Accuracy at different level of corruption
 - Accuracy drop (performance degradation)
- Visualise and discuss results



Data



Dataset Selection Criteria

- Type:
 - Image Classification
- Simplicity
 - Have to be relatively distinct classes
- Dataset Size
 - Multiple classes - Might want to investigate effect of various techniques on number of classes
 - Must contain a decent number of samples - Might want to investigate performance at various dataset size

Dataset Shortlist

- 1) Mammals Image Classification(45 Classes, 13571 images, 256x256):
<https://www.kaggle.com/datasets/asaniczka/mammals-image-classification-dataset-45-animals/data>
- 2) Insect Dataset (11 Classes):
<https://www.kaggle.com/datasets/ismail703/insects/data>
- 3) The Oxford-IIIT Pet Dataset (37 Classes, 200 image per class):
<https://www.kaggle.com/datasets/tanlikesmath/the-oxfordiiit-pet-dataset/data>

The Oxford-IIIT Pet Dataset

- Chosen because it is versatile - allow us to try both:
 - Binary Classification
 - Multi-class Classification
- Dataset properties:
 - File names are labelled "<class_name>_id"
 - Capitalised class name are cats, while non-capitalised class name are dogs
 - Annotation file given in ".txt" file

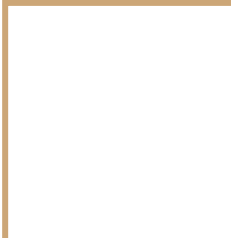
Annotation File

- Class ID (1 to 37)
- Species ID (1 and 2)
- Breed ID (1 to 25 and 1 to 12)
 - This is not needed for our experiments

```
#Image CLASS-ID SPECIES BREED ID
#ID: 1:37 Class ids
#SPECIES: 1:Cat 2:Dog
#BREED ID: 1-25:Cat 1:12:Dog
#All images with 1st letter as captial are cat images
#images with small first letter are dog images
Abyssinian_100 1 1 1
Abyssinian_101 1 1 1
Abyssinian_102 1 1 1
Abyssinian_103 1 1 1
Abyssinian_104 1 1 1
Abyssinian_105 1 1 1
Abyssinian_106 1 1 1
Abyssinian_107 1 1 1
Abyssinian_108 1 1 1
Abyssinian_109 1 1 1
Abyssinian_110 1 1 1
Abyssinian_111 1 1 1
Abyssinian_112 1 1 1
Abyssinian_113 1 1 1
Abyssinian_114 1 1 1
Abyssinian_115 1 1 1
Abyssinian_116 1 1 1
Abyssinian_117 1 1 1
Abyssinian_118 1 1 1
Abyssinian_119 1 1 1
Abyssinian_120 1 1 1
Abyssinian_121 1 1 1
Abyssinian_122 1 1 1
Abyssinian_123 1 1 1
```

Data Split

- Train-Val-Test
- Ratio Chosen (70-15-15):
 - Dataset is small
 - Evaluation is a key in our experiment



Training Setup



Ensuring Fair Comparison

- Use base version
 - Smallest version that exist for both MAE and standard ViT
- The only differences are:
 - metadata
 - num_classes
 - std and mean (Normalization)

```
-----Model Classes-----
<class 'timm.models.vision_transformer.VisionTransformer'>
<class 'timm.models.vision_transformer.VisionTransformer'>

-----Parameters-----
85800194
85800194
```

	Model 1	Model 2	Same?
url	https://dl.fbaipublicfiles.com/mae/pretrain/mae...		False
num_classes	1000	0	False
std	(0.5, 0.5, 0.5)	(0.229, 0.224, 0.225)	False
mean	(0.5, 0.5, 0.5)	(0.485, 0.456, 0.406)	False
license	apache-2.0	cc-by-nc-4.0	False
tag	augreg2_in21k_ft_in1k	mae	False
hf_hub_id	timmm/vit_base_patch16_224.augreg2_in21k_ft_in1k	timmm/vit_base_patch16_224.mae	False
input_size	(3, 224, 224)	(3, 224, 224)	True
fixed_input_size	True	True	True
interpolation	bicubic	bicubic	True
classifier	head	head	True
crop_mode	center	center	True
architecture	vit_base_patch16_224	vit_base_patch16_224	True
pool_size	None	None	True
first_conv	patch_embed.proj	patch_embed.proj	True
custom_load	False	False	True
crop_pct	0.9	0.9	True

Training Setup

- Learning Rate
 - Initial $1e-4$, Minimum $1e-6$
 - ReduceLROnPlateau scheduler (factor of 0.1)
- Early Stopping
 - Scheduler patience = 5
 - Overall patience = 10

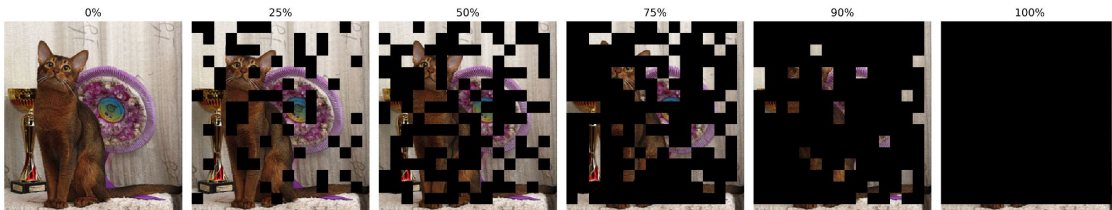


Corruption Transformations



Random Patch Mask

- Randomly mask out patches in the image
- mask_ratio refers to the percentage of patches to be masked



```
class RandomPatchMask:
    def __init__(self, patch_size=16, mask_ratio=0.75):
        self.patch_size = patch_size
        self.mask_ratio = mask_ratio

    def __call__(self, img):
        C, H, W = img.shape

        n_h = H // self.patch_size
        n_w = W // self.patch_size
        num_patches = n_h * n_w

        num_mask = int(num_patches * self.mask_ratio)

        mask_indices = torch.randperm(num_patches)[:num_mask]

        for idx in mask_indices:
            row = idx // n_w
            col = idx % n_w

            y1 = row * self.patch_size
            y2 = y1 + self.patch_size
            x1 = col * self.patch_size
            x2 = x1 + self.patch_size

            img[:, y1:y2, x1:x2] = 0

        return img
```

Gaussian Noise

- Add pixel variation based on the gaussian distribution
- std controls noise level

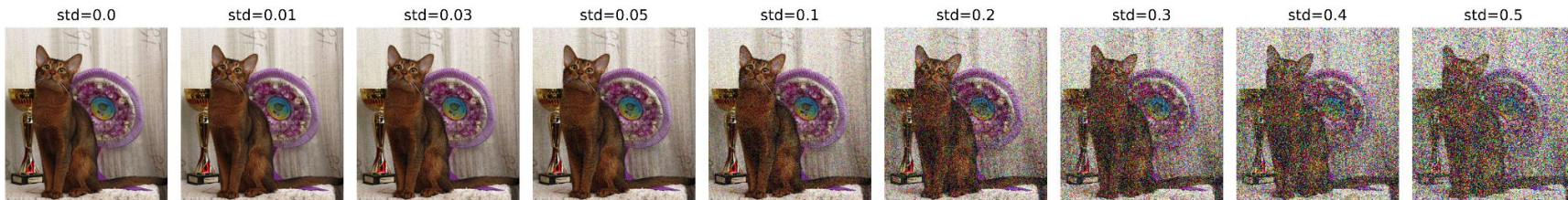
```
class GaussianNoise:
    def __init__(self, mean=0.0, std=0.05):
        self.mean = mean
        self.std = std

    def __call__(self, img):
        img = F.to_tensor(img)

        noise = torch.randn_like(img) * self.std + self.mean
        img = img + noise

        img = torch.clamp(img, 0., 1.)

        return F.to_pil_image(img)
```



Salt and Pepper Noise

- Impulse noise
- noise_ratio refers to percentage of pixels to corrupt
- salt_ratio refers to percentage of corrupted pixels to be white

```
class SaltAndPepperNoise:
    def __init__(self, noise_ratio=0.05, salt_ratio=0.5):
        self.noise_ratio = noise_ratio
        self.salt_ratio = salt_ratio

    def __call__(self, img):
        img = F.to_tensor(img)

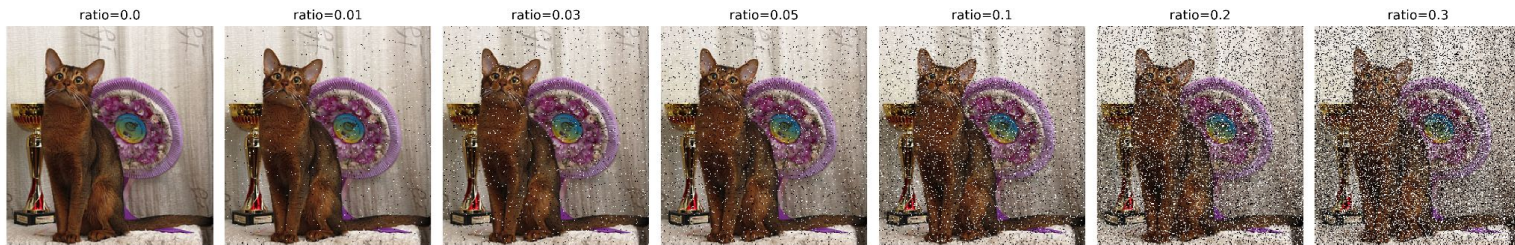
        C, H, W = img.shape

        noise_mask = torch.rand(H, W) < self.noise_ratio

        salt_mask = torch.rand(H, W) < self.salt_ratio

        img[:, noise_mask & salt_mask] = 1.0
        img[:, noise_mask & ~salt_mask] = 0.0

        return F.to_pil_image(img)
```



Cutout

- Similar but different with random patch mask
- Cutout refer to a continuous region rather separated patches

```
class Cutout:
    def __init__(self, percentage=0.25, fill=0.0):
        self.percentage = percentage
        self.fill = fill

    def __call__(self, img):
        img = F.to_tensor(img)

        _, h, w = img.shape

        cutout_h = int(h * self.percentage)
        cutout_w = int(w * self.percentage)

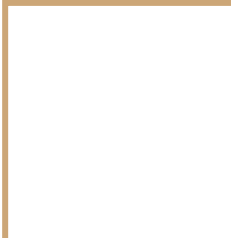
        cy = torch.randint(cutout_h // 2, h - cutout_h // 2, (1,)).item()
        cx = torch.randint(cutout_w // 2, w - cutout_w // 2, (1,)).item()

        y1 = cy - cutout_h // 2
        y2 = cy + cutout_h // 2
        x1 = cx - cutout_w // 2
        x2 = cx + cutout_w // 2

        img[:, y1:y2, x1:x2] = self.fill

    return F.to_pil_image(img)
```





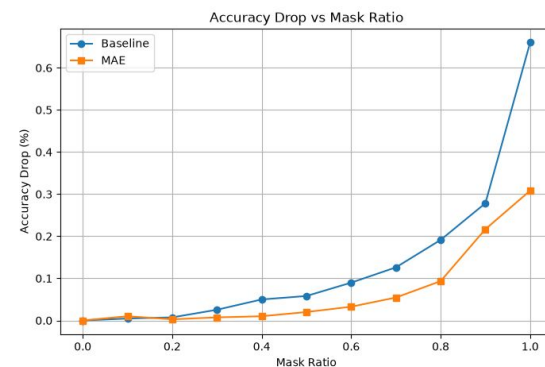
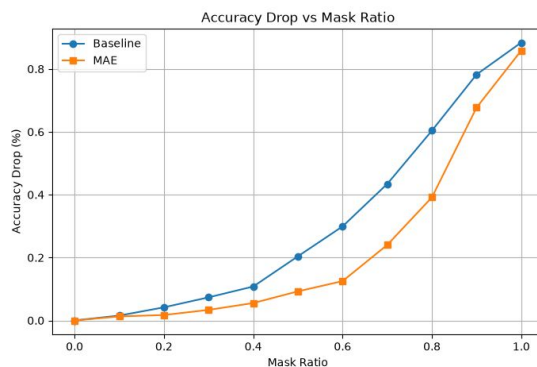
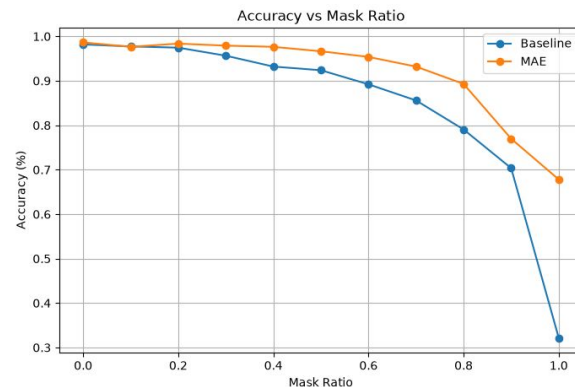
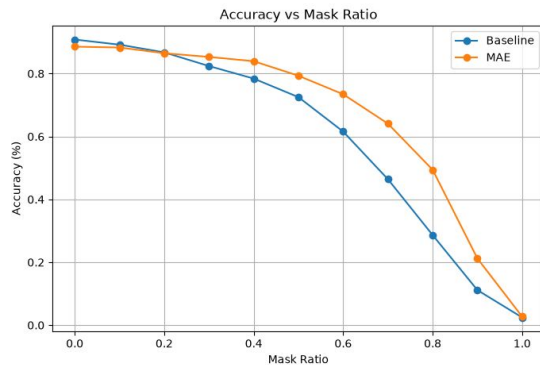
Results



Clean Test Set

- Binary Classification
 - Baseline ViT: 0.9818676337262012
 - MAE ViT: 0.986400725294651
- Multi-Class classification
 - Baseline ViT: 0.9084315503173164
 - MAE ViT: 0.885766092475068

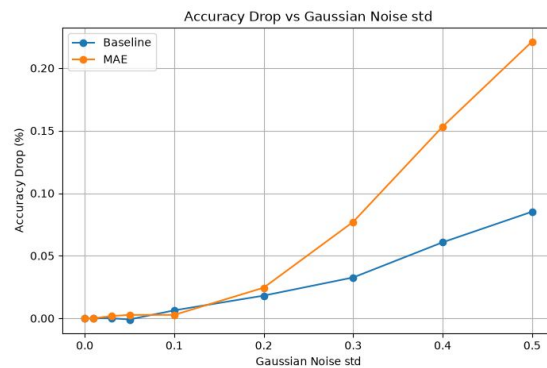
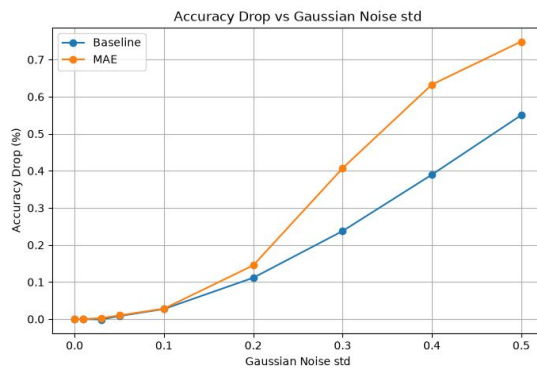
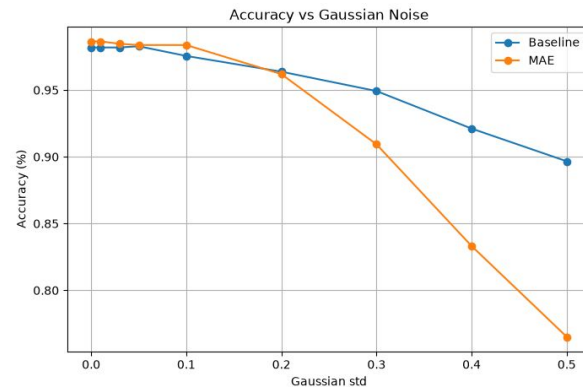
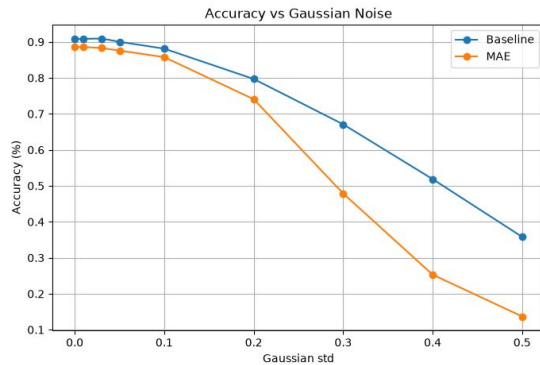
Random Patch Mask



Multi-Class Classification

Binary Classification

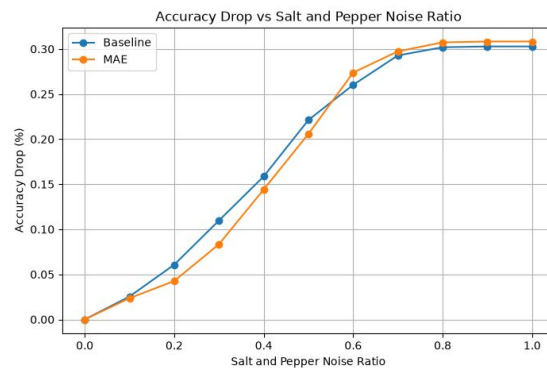
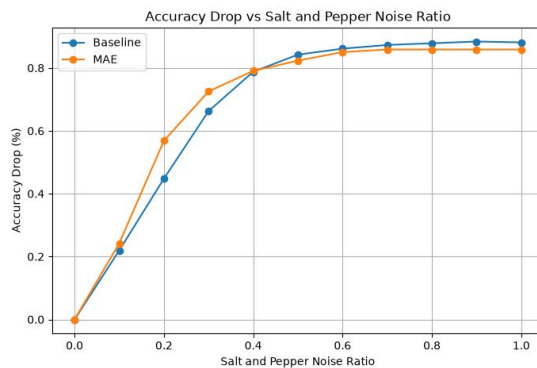
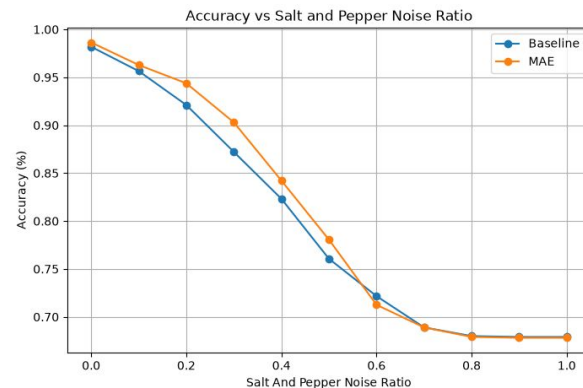
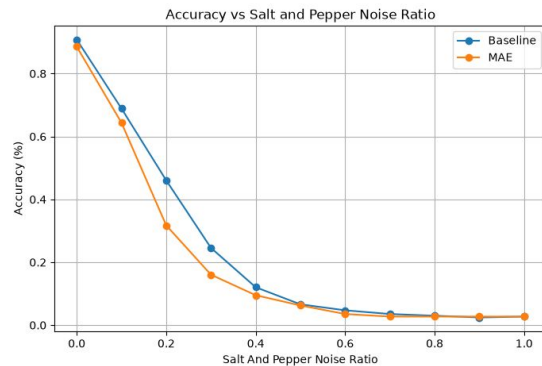
Gaussian Noise



Multi-Class Classification

Binary Classification

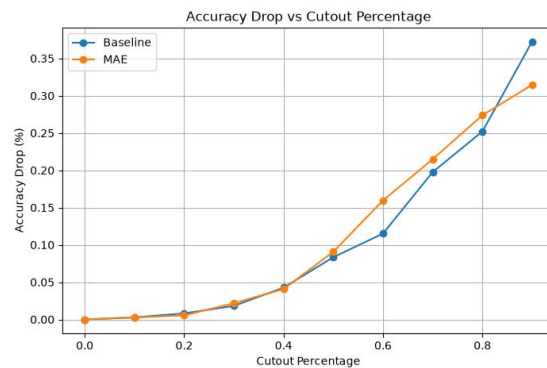
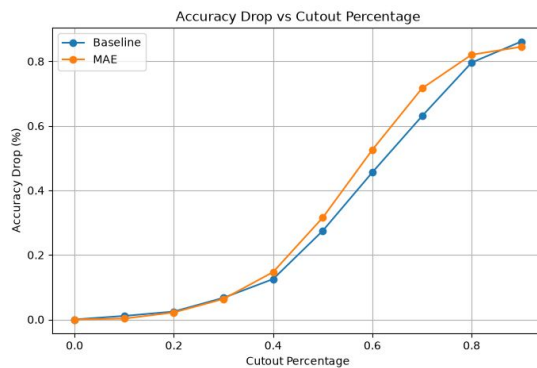
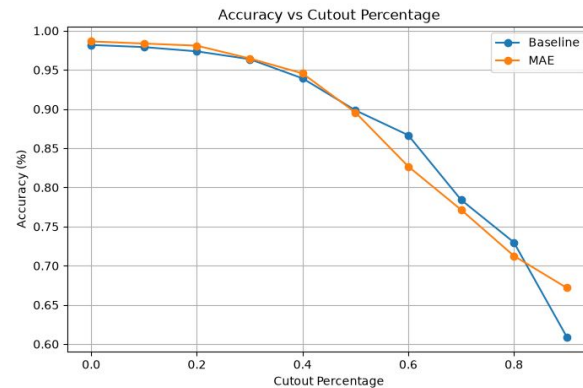
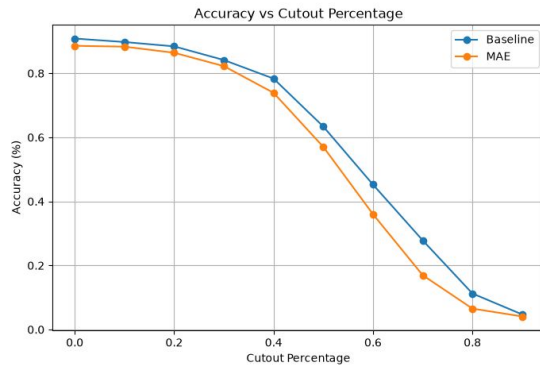
Salt and Pepper Noise



Multi-Class Classification

Binary Classification

Cutout

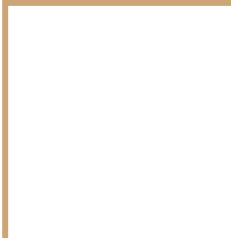


Multi-Class Classification

Binary Classification

Summary of Results

- All the noise types don't show MAE performing better except random patch masking
- Random patch masking shows obvious improvements with MAE especially at higher level of masking ratio
 - For binary classification, the accuracy of MAE is always higher
 - For multi-class classification, the accuracy of MAE starts lower but eventually overtakes the baseline ViT
 - The lower accuracy of MAE for multi-class classification may be explained by a more complicated classification layer
- Surprisingly, Cutout which is similar to random patch masking does not show improvement with MAE, this shows that having continuous region masked does not break the standard ViT like having random patches masked

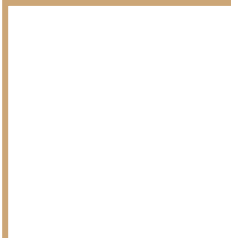


General Observations and Learning Points



General Observations and Learning Points

- Normalisation and std matters greatly
 - Performance drops when using a different normalisation and std, not by a lot but noticeable
 - Can't use the same transformation for both models
- Baseline ViT tend to converge earlier
 - Likely due to being trained with a classifier head, MAE does not come with a classifier head



Conclusion



Challenges

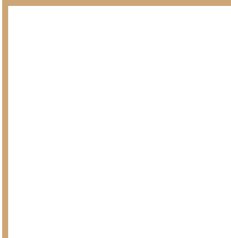
- Time Factor - Training is long:
 - Every slight mistake or bug means re-training and each training run takes hours to complete
- Difference in Normalization and std
 - Need different loader and transformations - complicating code
- Difference in model training optimizations
 - It's unclear what the optimal hyperparameters and training procedure for each model is

Limitation of Experiment

- Hardware limitations - Unable to work with pre-training:
 - Would have been interesting to see the pre-training effects of MAE
- Model availability:
 - ViT comes in many sizes but the MAE version only starts from ViT-Base
 - Using a different sized model would be unfair for comparison so the smallest model is Base
 - Base-sized models are already very good for the simple task like ours, so the effects is hard to see
- Difference in Normalization and std
 - Need different loader and transformations - complicating code
- Difference in model training optimizations
 - It's unclear what the optimal hyperparameters for each model is

What Could Be Better?

- Time management
 - Should have worked on report earlier instead of trying to push for good results
- Code clarity
 - Honestly could have been better especially with the dataloader and transformations, there's a lot of repeatable code
- Figure saving
- Doing linear probing instead of end to end fine tuning
 - end to end fine tuning is simpler but may cause catastrophic forgetting which hurts the result by removing the advantages of what MAE learned
- A smaller model might show the effect of MAE better
 - However, MAE only released the base version



Thank You



Citations

- 1) Parkhi, O. M., Vedaldi, A., Zisserman, A., & Jawahar, C. V. (2012). Cats and dogs. 2012 IEEE Conference on Computer Vision and Pattern Recognition, 3498–3505. IEEE.
- 2) He, K., Chen, X., Xie, S., Li, Y., Dollár, P., & Girshick, R. (2022). Masked autoencoders are scalable vision learners. Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 16000–16009.

Github Repo: <https://github.com/IAmGreyBunny/93478-Computer-Vision>